

Stationary Shop CRUD Manager — Project Statement

****Student Name:** Rashi Agarwal ****Roll Number:** 25bcy10248 ****Course:** flipped******

1. Project Title ****Stationary Shop CRUD Manager****

2. Project Overview The Stationary Shop CRUD Manager is a simple full-stack application designed to manage the day-to-day operations of a small stationery shop. It provides Create, Read, Update, and Delete (CRUD) functionality for products, customers, and orders. The project aims to demonstrate core concepts of database design, RESTful APIs, and a basic front-end interface for inventory and sales management.

3. Objectives - Build a reliable CRUD system for managing stationery products. - Implement searchable and paginated lists for inventory. - Allow record creation (add new products/customers), updates (edit product details), and deletion (remove discontinued items). - Track simple orders and generate basic sales reports. - Demonstrate best practices in project structure, documentation, and testing.

4. Features - Product management: add, view, edit, delete products. - Category support for products (e.g., pens, notebooks, art supplies). - Customer management: add and manage customer data. - Order management: create orders, list orders, view order details. - Simple stock tracking: update quantity on order placement. - Search and filter for products and orders. - Responsive front-end (basic) and RESTful backend API.

5. Technologies & Tools - Frontend: HTML, CSS, JavaScript (optionally a framework such as React or Bootstrap for styling). - Backend: Node.js with Express ****OR**** Python Flask (anyone may be chosen based on preference). - Database: SQLite / MySQL / PostgreSQL for persistent storage. - Development tools: Git, VS Code, Postman for API testing. - Optional: ORM (Sequelize for Node.js, SQLAlchemy for Python).

6. System Design & Database Schema (suggested) **Tables:******

- 1. ``products`` - ``id`` (PK)
- ``name`` - ``category`` - ``price`` - ``stock_quantity`` - ``description``

- 2. ``customers`` - ``id`` (PK) - ``name`` - ``phone`` - ``email`` - ``address``

- 3. ``orders`` - ``id`` (PK) - ``customer_id`` (FK → `customers.id`) - ``order_date``
- ``total_amount``

- 4. ``order_items`` - ``id`` (PK) - ``order_id`` (FK → `orders.id`) - ``product_id`` (FK → `products.id`)
- ``quantity`` - ``unit_price``

7. API Endpoints (example, RESTful) - ``GET /api/products`` — list products (supports search, pagination) - ``GET /api/products/:id`` — get product details - ``POST /api/products`` — create a product - ``PUT /api/products/:id`` — update a product - ``DELETE /api/products/:id`` — delete a product

- `GET /api/customers` — list customers - `POST /api/customers` — add customer
- `GET /api/orders` — list orders - `POST /api/orders` — create order (with order items) -
`GET /api/orders/:id` — order details

8. Installation & Setup (example commands) 1. Clone the repository `git clone <repo-url>`

2. Backend (Node.js example) ``` cd backend npm install npm run migrate # if migrations are configured npm start ```

3. Frontend ``` cd frontend npm install npm start ```

4. Open the frontend at `http://localhost:3000` (or the configured port).

9. Usage Instructions - Add initial products via the UI or API. - Create customers or add them at the time of placing an order. - Place orders — when an order is placed, stock quantities should be decremented. - Use search and filters to quickly find products. - Delete products carefully — include a confirmation step in the UI.

10. Testing - Unit tests for backend routes and database operations. - Integration tests for key flows: adding product → creating order → stock adjustment. - Manual testing with Postman for API endpoints.

11. Screenshots & Demo *(Add screenshots here of the frontend pages: product list, add product form, order creation page, and report page.)*

12. Future Enhancements - Role-based authentication (admin/seller/viewer). - Barcode scanning for fast product lookup. - Sales analytics dashboard (daily/weekly/monthly sales). - Export reports to CSV / PDF. - Mobile-friendly app or progressive web app (PWA).

13. References - Official documentation for chosen backend and frontend frameworks. - Tutorials and sample projects demonstrating CRUD operations and REST APIs.

14. Conclusion This project provides a compact but complete demonstration of building a CRUD application for a stationery shop. It is suitable for a semester project and can be expanded with additional features (auth, reporting, analytics) as required.

****Prepared by:** Rashi Agarwal ****Roll Number:** 25bcy10248 ****Course:** flipped******